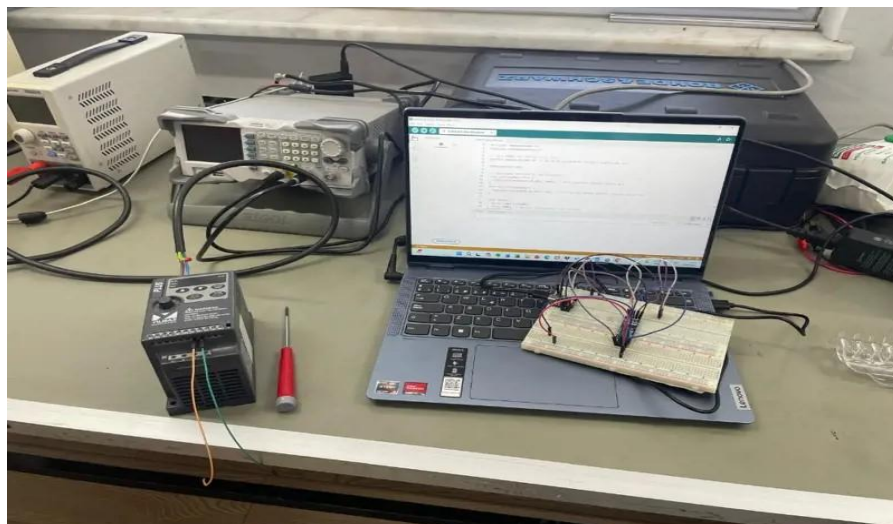
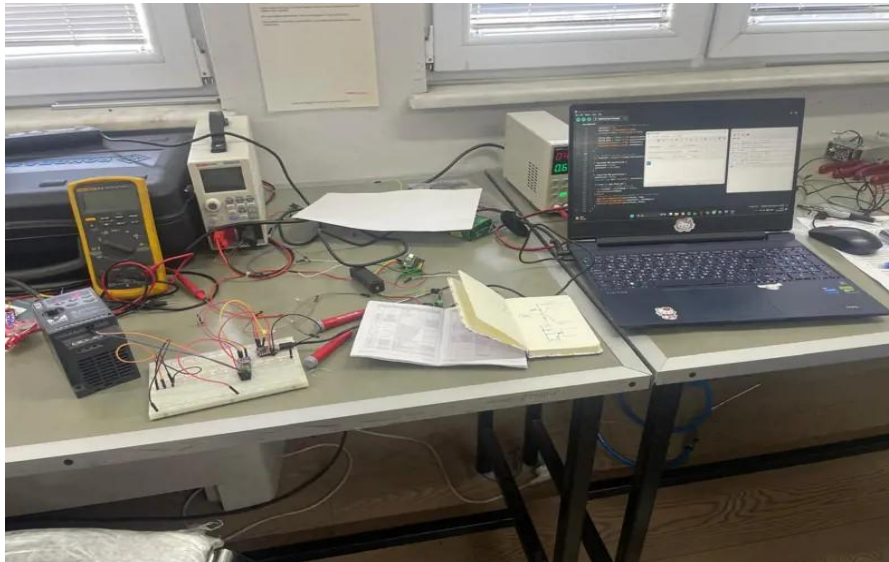


The design of an independent embedded system project has been initiated with the aim of delving deeply into the software-hardware architecture underlying electronic systems. The main objective of the project is the creation of the portable “Parameter Loading Device” which will be able to load pre-designed parameters from its internal memory to industrial drives with one single move, regardless of whether there is a computer or an additional interface present on the ground. At the first stage of the project, the system architecture, hardware needs, and communication network were defined. For that reason, the ESP32-C3 microcontroller was used due to its small size, sufficient flash memory capacity, and high-speed operation. In order to transmit logic signals coming from the microcontroller to differential signals, RS-485 transceiver IC was installed. Following the definition of the components and the creation of the schematic, prototyping continued with the configuration of the basic hardware on the breadboard inside the laboratory setting. After the construction of the hardware part of the device was done, software programming proceeded according to Modbus RTU standard in order to create communication with the drive.

The ESP32-C3 microcontroller was programmed with the help of C/C++ codes created using the integrated development environment as “Master” for the communication line. With the help of connecting the RS+ and RS- ports of the microcontroller with the motor driver, reading commands were sent to the particular address of the registers on the driver. The analysis of the information transmitted through the serial port monitor proved that the information (holding registers) from the drive was received entirely.

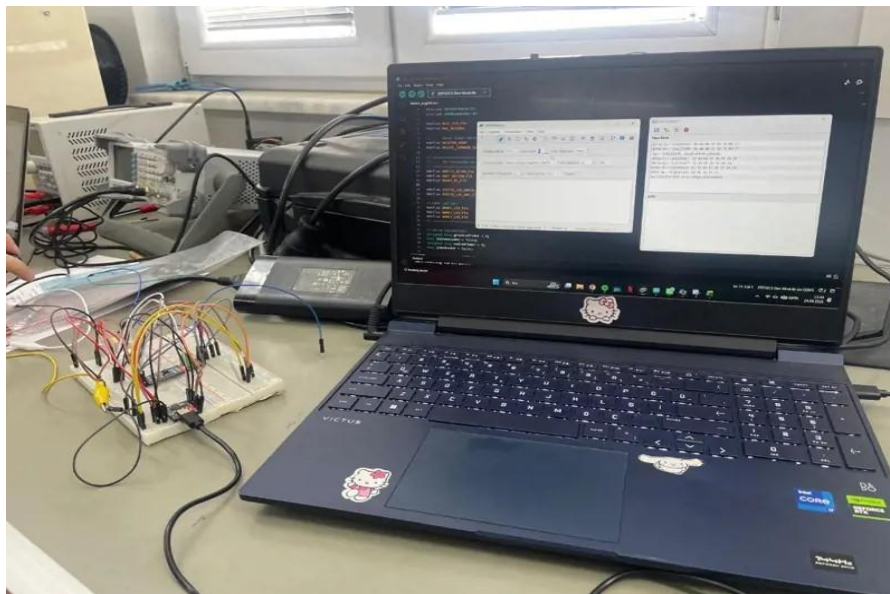


Following the installation of the hardware infrastructure the previous day, the “Parameter Loading Device” project has moved on to the software validation and communication analysis phases. To check whether the created ESP32-C3 system can send Modbus RTU frames to the motor driver, and then to parse its register map, the QModMaster software, the open-source simulator and test application, was used on the system computer. The RS-485 bridge between the computer and the driver was provided by the breadboard configuration in the laboratory environment. At first, through the software interface, the physical layer parameters (the baud rate, data bits, stop bits, and parity) were set according to the driver's defaults. After that, the addressing was performed by assigning the Slave ID to the driver on the network. Once the connection was established, the access to the driver's memory addresses was provided via QModMaster software. Basic parameters such as the motor start/stop timing and maximum frequency reference were read through the "Read Holding Registers" command, and the values were changed using the "Write Single Register" command in the software interface and confirmed on the driver's LCD display. In particular, with this type of analysis, the raw hexadecimal TX and RX data packets could be seen in the "Bus Monitor" tab of QModMaster. This analysis helped to assess how the hardware and software, especially the C/C++-based Modbus library to be developed for the ESP32-C3, will independently create these data packets.

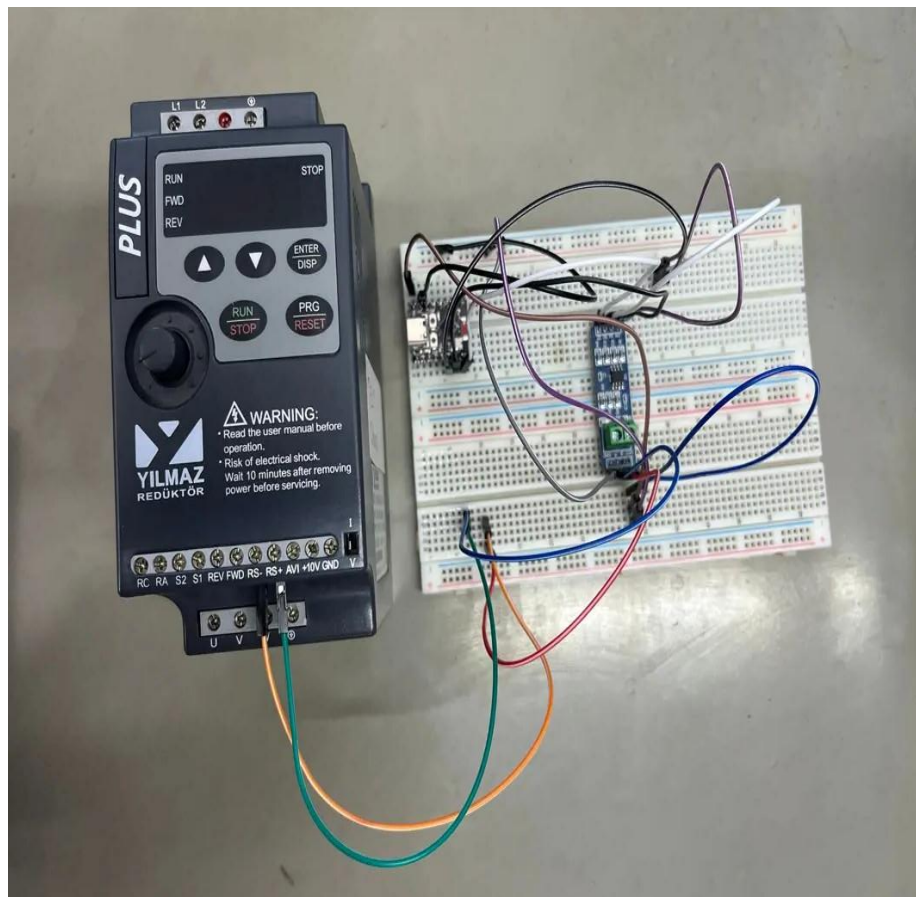


Progress in development of the “Parameter Loading Device” project based on ESP32-C3 microcontroller has taken place with a focus on hardware feedback systems and software data handling management. As the target of the project is autonomous functioning in the field without using a computer (PC) and any kind of display, hardware has been developed including LED status indicators, showing states like "Successful Read/Write," "Communication Error," and "System Ready." The LEDs have been placed in the breadboard circuit, connected to ESP32-C3 GPIO pins, and programmed to operate under certain logical conditions.

The most important software feature that has been added at this stage is the expansion of the data management system to improve flexibility of operations. Even though the previous version was focused on a single device and a single data set, now the architecture has been adjusted to a "3 Bank, 3 Slave, 3 Memory" architecture to allow usage in different kinds of motors and machines in industrial settings. The adjustment in the structure allowed adding multidimensional arrays of data, containing three parameter banks memories. The device has been improved to read parameters from three different memories in the RS-485 bus, pick one of the chosen parameter groups and load them into the corresponding device.

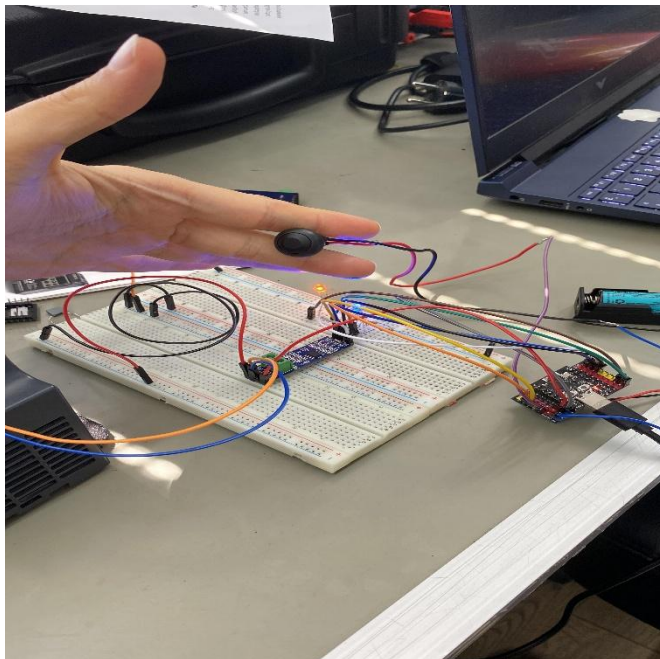


The operational specifications for the project titled "Parameter Loading Device" based on the ESP32-C3 chip have been specified, and management of persistent memory storage, which has been recognized as a vital stage within the software design, has been put in place. At the beginning of the day, the functional specifications of the project (input buttons, LED feedback matrix, error processing, and Modbus timeout periods) have been identified and put down in the engineering specification. The major risk inherent within the software design has been the possible loss of parameter banks, which have been saved in the memory during the cycle when the device has been turned off and then on again. In order to prevent this risk, an emulated EEPROM (Preferences) space has been created within the ESP32-C3 microcontroller internal Flash memory. The feature built in the code developed in the C/C++ language allows saving the selected bank, slave addresses, and the custom parameter sets in the non-volatile memory. Tests proved that the saved parameters persist even after the complete cut-off of the power supply and its restart and can be uploaded into the driver using the RS-485 bus.



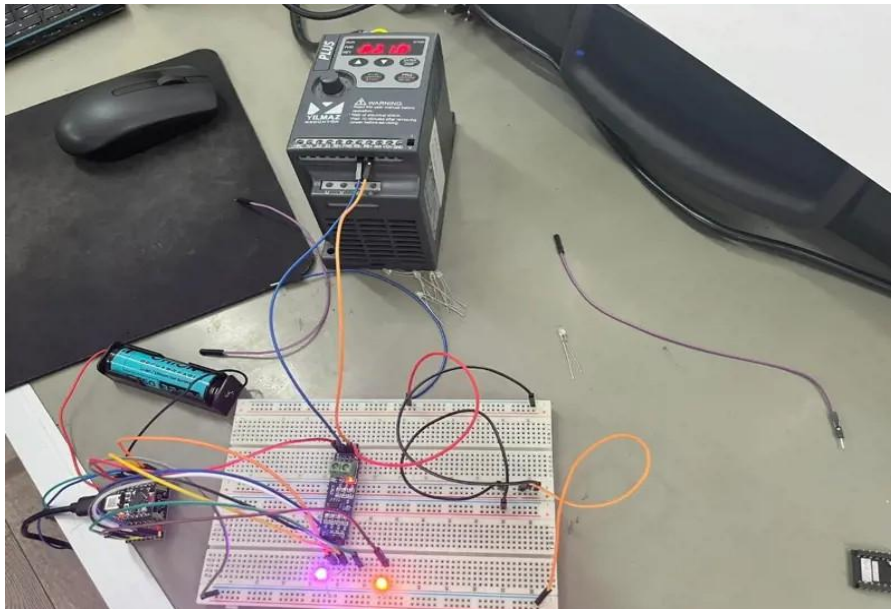
The human-machine interface (HMI) designed for the ESP32-C3 based "Parameter Loading Device" project has been fully completed, and work is ongoing in the area of mechatronics applied to factory production lines. As part of the current project, a physical control button has been added to the mobile device that is being developed. The button has been programmed using software with regard to its optimal use in the field: one single tap of the button triggers the loading process of the pre-stored parameter set to the driver via the RS-485 Modbus bus; holding the button toggles between preset parameter banks, thus selecting various motor/application parameter sets. This button program was programmed and tested in laboratory conditions.

In the later part of the day, attention was paid to coupling drives to asynchronous motors to strengthen practical skills related to production lines. The drive unit has been physically and electrically coupled to the terminal block of an industrial three-phase asynchronous motor. The attachment process of the drive to the motor included connecting the stator winding terminals to the corresponding terminals and insulating them, as well as connecting the PE to the housing.



Hardware implementing the functionality of power management has been implemented within the framework of the ESP32-C3-based “Parameter Loading Device” project, which is being developed, in order to provide the portability and energy autonomy (independent working of the device). In order to eliminate the necessity of using the external adapter or computer cable in the laboratory test bench, a rechargeable Li-ion battery has been included as the internal power supply source. The power management module expansion board has been implemented in the hardware to make sure that the device is able to be supplied by the battery when working in the field and to be charged safely when connected to the computer (when uploading code).

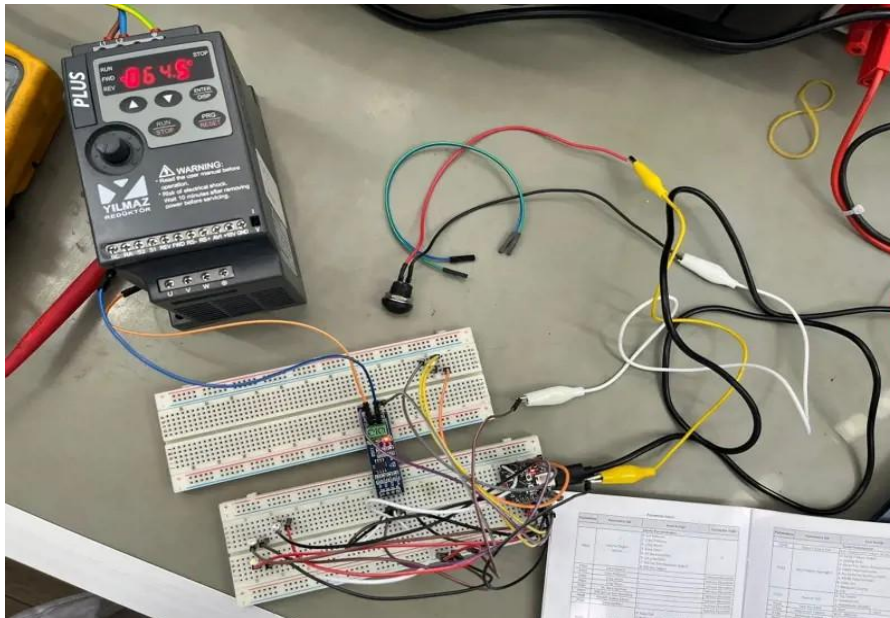
The implemented charging circuit and voltage regulators, it becomes possible to use the 5V power supply provided through the USB port both for powering the microcontroller and for charging the Li-ion battery. The rechargeable power management system has been tested on the breadboard and it has been proved that the device was able to perform autonomous communication with the motor drivers via the RS-485 Modbus bus regardless of power interruptions. Moreover, the code of the system has been optimized according to the clean code approach.



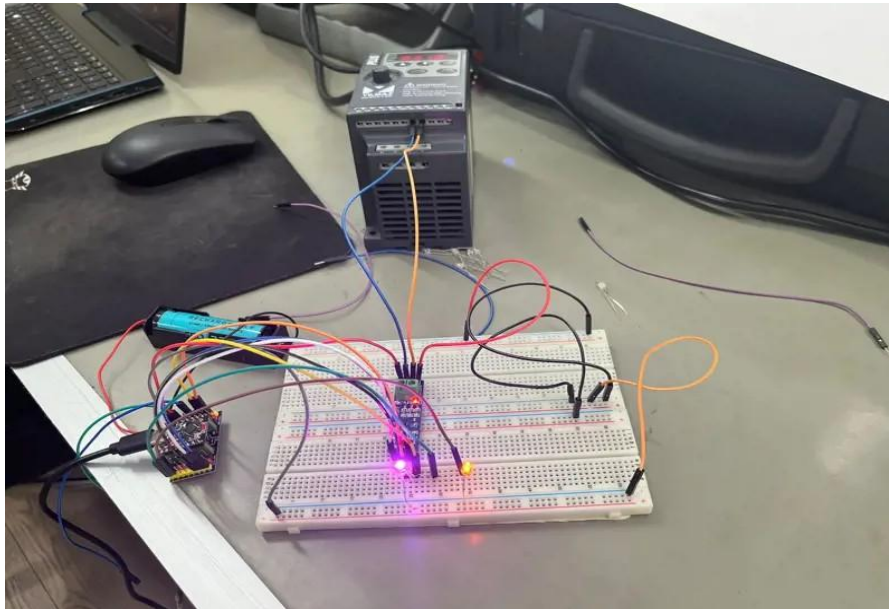
The development of the Parameter Loading Device based on the ESP32-C3 module is ongoing at the moment. As part of the project, the testing process of parameter writing capabilities in the driver has been started, which is a crucial step of the process. After preliminary tests of the software, the transmission of data packets using the Modbus RTU protocol and processing of these commands by the driver has failed. In order to resolve the problem, the code was simplified through eliminating the unnecessary loop and changing the timing parameters, such as timeouts and delays. The subsequent modifications in the software were accompanied by using the handheld digital oscilloscope available in the laboratory in order to check the quality of hardware signals. The oscilloscope probes were connected to the RS-485 transceivers (RS+ and RS-) in order to observe the transition of differential signals, logical levels during the moment of packet transmission, as well as any possible noise and interference phenomenon. Through debugging of both the software and hardware parts of the system, its stability was achieved. During this experiment, the parameter writing into the driver memory using one run of the developed device was successfully achieved.



Optimization and debugging processes have been contributing greatly to the improvement of the software stability of the ESP32-C3 based "Parameter Loading Device" project. With a view to solving system-wide software problems, the RS-485 Modbus communication loops were analyzed; the duration of debug operations and timeout settings were configured anew in order to prevent any time delay in the process of transmitting and receiving data packets. Besides that, the necessary changes were made in the communication library to assure the proper data transfer in the system. The next sphere where integration of the software and hardware has been performed concerns the management of non-volatile memory (Flash/Preferences). A procedure was introduced which allows users to reset or clear parameter banks saved in the memory of the device. This feature works via pressing the physical BOOT button on the ESP32-C3 board. When the user presses this button while powering the device or keeps it pressed for the necessary period of time, all parameter records in the selected non-volatile memory bank are wiped out (factory reset).



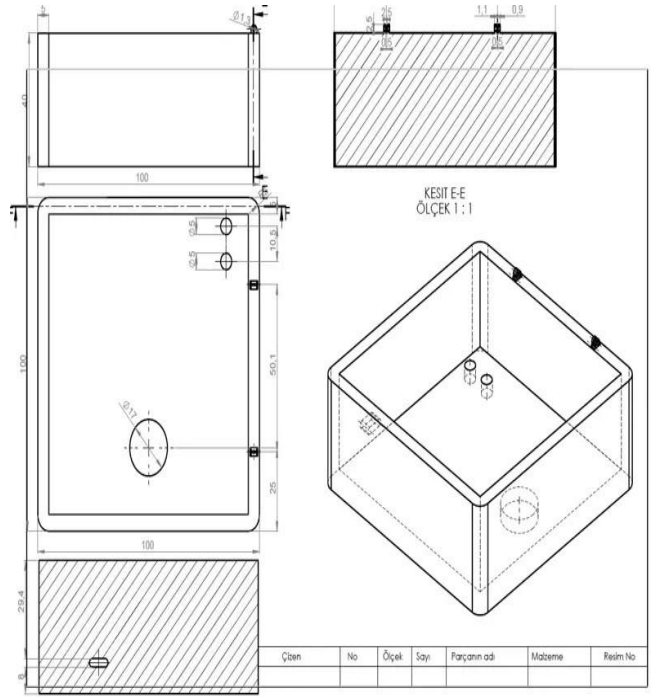
The architecture of the "Parameter Loading Device" project based on ESP32-C3 microcontroller has received further development through implementation of arrays for data structuring. The following conclusion has been reached that some parameters in driver configuration of the industry must assume different values in order for them to be activated (e.g., necessity of sending the values "1" and then "0" in sequence to activate or reset a function). In order to solve this task, the code has been improved with application of arrays functionality in the C/C++ programming language. As a result of the development of the algorithm, target parameters and their values in configuration have been defined as arrays in logical order. In the case of command transmission via the RS-485 Modbus bus, the device transmits sequential write commands to the driver in the order of arrays processing. As the result of laboratory tests, the situations when a parameter must first assume the value 1 and then 0 have been simulated, and it has been proven with the help of driver's display that the parameters have been processed in sequence.



Mechanical design work has begun to transition the ESP32-C3-based “Parameter Loading Device” project from the prototype stage to its final product form . In the first part of the day, a custom protective enclosure design capable of safely housing the developed electronic components (ESP32-C3, expansion board, Li-ion battery, buttons, and connection terminals) was created using SolidWorks. During the design process, the enclosure’s external dimensions, and component layouts were optimized; the openings and clearance gaps required for the device to communicate with the outside world (USB charging/programming port, cable outlets, and button/LED interfaces) were drawn in accordance with technical drawing standards.

In parallel with the mechanical design, optimizations were made to the device’s software and logic structure for error handling. Intelligent warning mechanisms were added to the software to enhance user safety and prevent potential configuration errors.

In this context, optimization code has been integrated into the software to ensure that the system detects situations where limits are exceeded such as when more than 100 parameters are assigned to a single memory bank during memory bank management and triggers a “red flash” to visually alert the user.



The final version (V1.0) of the project titled “Parameter Loading Device” based on ESP32-C3 microcontroller has been finalized after performing all necessary optimizations. At the beginning of the day, there were performed performance tests of the device; timing performance was assessed by measuring time required for transferring a set of parameters to the driver using the RS-485 Modbus RTU bus. Optimization of the project user interface and its error handling module was implemented. In the course of optimization, the following logical improvements were incorporated into the software and RGB LED feedback matrix: error codes corresponding to different kinds of errors (communication, timeout, wrong register address) have been defined and updated. In order to make it easier for the user to identify the active parameter bank on the device, colors of banks were assigned using RGB representation.

The “Green Flash” (green flashing) is used to notify the user of successful completion of the parameter loading procedure, while “Red Flash” (red flashing) indicates violation of the limits and communication errors. With the successful execution of software and hardware testing, the project was completed. We did the presentation of the project and made our mentor aware of the progress of the project. Our project was completed as we were sure that there was no need for any further requirement in first version (V1.0).

